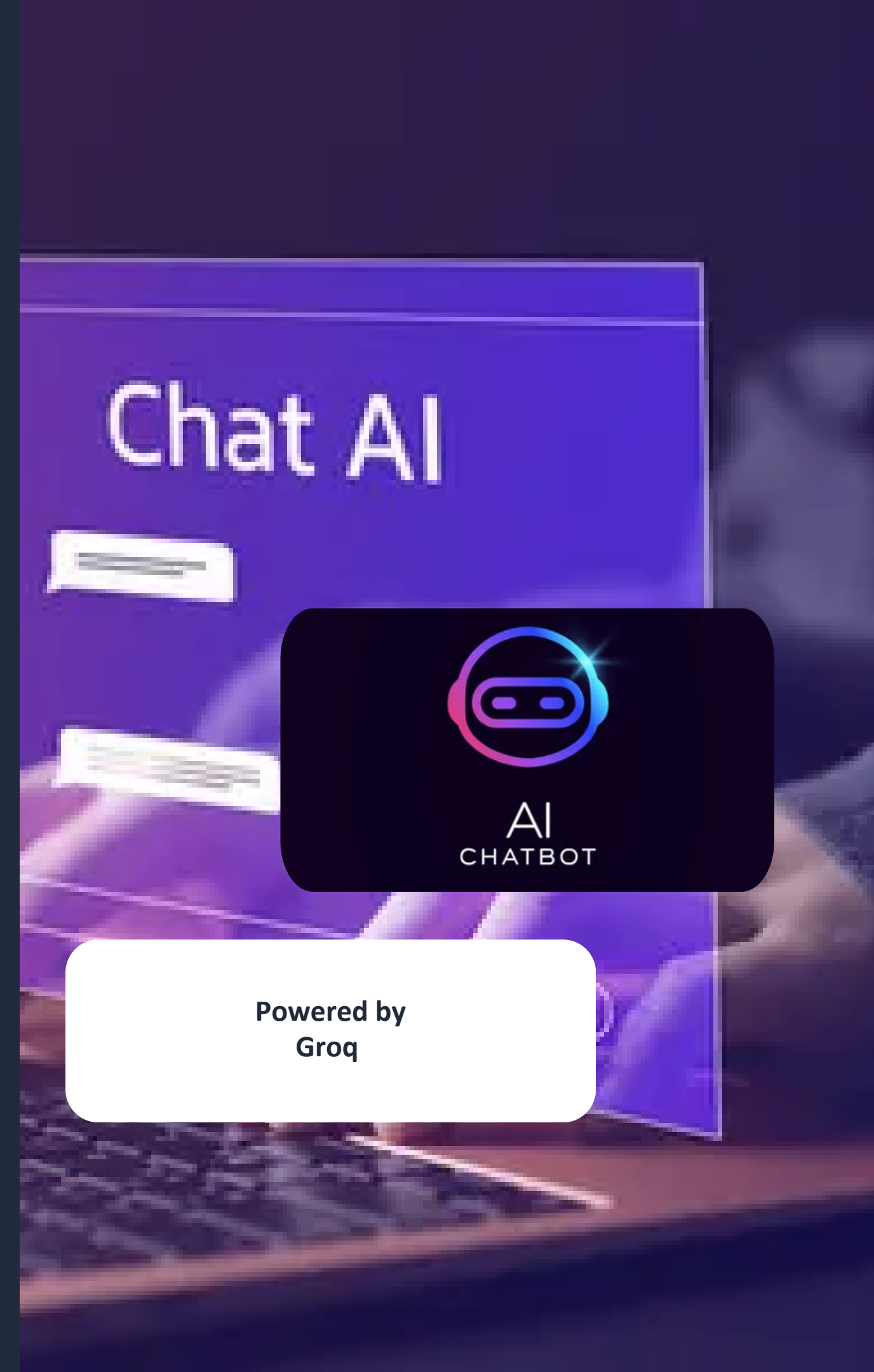


JAIN UNIVERSITY

AI Student Help Desk

A conversational assistant that answers admissions, fee, and campus questions instantly — with optional voice input.

July 2026



Our Team

Five members, five areas of ownership

1

Member 1

BRAHMADEVU HRITHIN
KUMAR

23BTRCC010

2

Member 2

Belljer.B.L

23BTRCC007

3

Member 3

Adithya dev DJ

23BTRCA023

4

Member 4

Buvanash

23BTRCC009

5

Member 5

Chirayu Bangera

23BTRCA021

Agenda

What we'll walk you through today

01

Problem & Objectives

Why students needed this, and what we set out to build

02

System Overview

How a conversation flows end to end

03

Architecture & Tech Stack

APIs, browser tech, and the request pipeline

04

Key Features

Guardrails, quick chips, voice input, resets

05

Model Variants

Groq (Llama) build vs. Gemini build

06

Challenges, Roadmap & Team

What we learned and what's next

The Problem

Prospective and current students ask the same questions over and over — admissions staff can't be available 24/7 across every channel.

1

Repetitive queries

Fees, eligibility, hostel & placement questions flood every helpdesk channel.

2

Limited office hours

Admissions staff can't answer instantly outside working hours.

3

Inconsistent answers

Different staff members can quote slightly different figures.

4

No self-serve option

Students must call, email, or visit campus just to ask a simple question.

Objectives

What the assistant needed to achieve

1

Instant answers

Give students 24/7 responses on admissions, fees, courses and hostel life.

2

Stay on-topic

Restrict every answer strictly to a curated Jain University knowledge base.

3

Multiple input modes

Support both typed and spoken questions for accessibility and speed.

4

Compare AI providers

Prototype the same assistant on two LLM backends — Groq and Gemini.

How a Conversation Flows

From a typed or spoken question to a grounded answer



Architecture & Tech Stack

A lightweight, client-only build — no backend server required

1

Front-end

HTML5 · CSS3 · Vanilla JavaScript

Single-page chat UI, no framework or build step

2

Voice layer

Web Speech API (SpeechRecognition)

Free, browser-native mic-to-text (Chrome / Edge)

3

Intelligence layer

Groq Chat Completions (Llama / gpt-oss-120b) · Google Gemini API

System prompt + hard-coded knowledge base ground every reply

Key Features

What makes the help desk feel considered, not just functional



In-browser API key

Key is entered once, kept client-side only — never sent to a third-party server.



Quick-question chips

Tappable suggestions (fees, hostel, placements) speed up common asks.



Domain guardrail

Off-topic questions get a polite, consistent redirect instead of a guess.



Link filtering

URLs typed into the chat are blocked with an inline warning.



Voice input

Mic button uses the Web Speech API to transcribe spoken questions live.



Retry with back-off

Groq build auto-retries on 429/503 with exponential back-off.

What the Bot Actually Knows

A hard-coded knowledge base covering admissions, fees, hostel & placements

#62

NIRF 2025 University rank

21,000+

Students, 50+ countries

500+

Programmes across 17 streams

~90%

Engineering placement rate

₹8 LPA

Average placement package

3,000+

Recruiting companies

Also covers: JET entrance exam, scholarships (Medha, Eklavya...), campus locations & contact details.

Feature Spotlight: Voice Input

Letting students speak their question instead of typing it



Web Speech API

Uses the browser's built-in SpeechRecognition — no extra SDK or paid transcription service.



One-tap mic button

Tapping the mic starts listening; the transcript fills the text box automatically.



Graceful fallback

Detects unsupported browsers (e.g. Brave, which blocks the speech backend) and disables the mic instead of failing silently.



Zero added cost

No extra API key or billing — speech-to-text runs entirely client-side.



"When is the JET exam?"

● Listening...

Chrome / Edge supported

Two Model Variants, Same Experience

We built and compared the assistant on two different LLM backends

Backend	Groq Cloud	Google Gemini
Model	openai/gpt-oss-120b (Llama-family)	gemini-3.5-flash
Endpoint style	OpenAI-compatible /chat/completions	generateContent, systemInstruction field
Auth header	Authorization: Bearer <key>	x-goog-api-key: <key>
Message roles	system / user / assistant	user / model (remapped)
Reliability	Exponential back-off retry on 429 / 503	Single attempt, surfaced error on failure

Challenges We Solved

Real problems that shaped the build



Model going off-topic

Strict system-prompt rule forces a fixed refusal line for anything outside the Jain University knowledge base.



Rate limits & server load

Groq build retries automatically with exponential back-off before showing an error.



Browsers without speech support

Feature-detects the Web Speech API and disables the mic button gracefully (e.g. on Brave) instead of crashing.



Users pasting links / spam

Client-side regex blocks URLs before they're ever sent to the model, with a visible inline warning.

Future Scope

Where this project could go next

01

Secure backend proxy

Move the API key server-side so students never handle it directly.

02

Voice output (TTS)

Add spoken replies for a fully hands-free experience.

03

Live data sync

Pull fees, seats & deadlines from a CMS instead of a hard-coded prompt.

04

Multi-language support

Kannada / Hindi input and output for wider accessibility.

05

Analytics dashboard

Track common questions to improve the knowledge base over time.



Thank You

Questions & Answers

JAIN University AI — Student Help Desk
Built with Groq · Gemini · Web Speech API